



LeetCode 167 — Two Sum II (Input Array Is Sorted)

1. Problem Title & Link

Title: LeetCode 167: Two Sum II — Input Array Is Sorted

Link: <https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/>

2. Problem Statement (Short Summary)

Given a **sorted array** of integers and a target value, find two numbers such that:

$\text{numbers}[i] + \text{numbers}[j] = \text{target}$

Return their indices (1-based indexing).

The problem guarantees exactly one solution.

3. Examples (Input → Output)

Example 1

Input:

numbers = [2,7,11,15]

target = 9

Output:

[1,2]

Explanation:

$2 + 7 = 9$

Example 2

Input:

numbers = [2,3,4]

target = 6

Output:

[1,3]

Example 3

Input:

numbers = [-1,0]

target = -1

Output:

[1,2]

4. Constraints

$2 \leq \text{numbers.length} \leq 3 * 10^4$



$-1000 \leq \text{numbers}[i] \leq 1000$

numbers is sorted in non-decreasing order

Exactly one solution exists

Important:

Array is already sorted

This hint is the key to the optimal solution.

5. Core Concept (Pattern / Topic)

Two Pointers

This is one of the most important Two Pointer interview problems.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

Check every pair.

for i

for j

if $\text{numbers}[i] + \text{numbers}[j] == \text{target}$

Example:

[2,7,11,15]

Check:

2+7

2+11

2+15

7+11

...

Time Complexity:

$O(n^2)$

Too slow.

Better Approach (HashMap)

Store visited elements.

target - current



lookup in map.

Time:

$O(n)$

Space:

$O(n)$

Works but doesn't use the sorted property.

Optimal Thinking

Since the array is sorted:

[2,7,11,15]

Use two pointers.

L = 2

R = 15

Sum:

17

Too large.

Move right pointer.

Why?

Because:

Array is sorted.

Moving right pointer left decreases sum.

Moving left pointer right increases sum.

This allows us to find the answer in one pass.

7. Visual / Intuition Diagram (ASCII)

```
numbers
```

```
[2, 7, 11, 15]
```

```
  L           R
```

```
sum = 17
```

```
Too large.
```

```
Move R left.
```

```
[2, 7, 11, 15]
```

```
  L           R
```

```
sum = 13
```

```
Still large.
```



```
Move R left.  
[2,7,11,15]  
  
L R  
  
sum = 9  
Found answer.
```

8. Pseudocode

```
left = 0  
right = n-1  
  
while left < right  
  
    sum = numbers[left] + numbers[right]  
  
    if sum == target  
        return indices  
  
    if sum < target  
        left++  
  
    else  
        right--
```

9. Code Implementation

Python

```
class Solution:  
    def twoSum(self, numbers, target):  
  
        left = 0  
        right = len(numbers) - 1  
  
        while left < right:  
  
            current = numbers[left] + numbers[right]  
  
            if current == target:  
                return [left + 1, right + 1]  
  
            elif current < target:  
                left += 1  
  
            else:  
                right -= 1
```

**Java**

```
class Solution {  
  
    public int[] twoSum(int[] numbers, int target) {  
  
        int left = 0;  
        int right = numbers.length - 1;  
  
        while(left < right) {  
  
            int sum = numbers[left] + numbers[right];  
  
            if(sum == target) {  
  
                return new int[]{left + 1, right + 1};  
  
            } else if(sum < target) {  
  
                left++;  
  
            } else {  
  
                right--;  
  
            }  
        }  
  
        return new int[]{};  
    }  
}
```

10. Time & Space Complexity**Time Complexity**

$O(n)$

Each pointer moves at most n times.

Space Complexity

$O(1)$

Only variables are used.

11. Common Mistakes / Edge Cases**Mistake 1**

Returning 0-based indices.

Wrong:

[0,1]



Correct:

[1,2]

Mistake 2

Using nested loops.

$O(n^2)$

Not needed.

Mistake 3

Moving the wrong pointer.

If sum is small:

Move left

If sum is large:

Move right

12. Detailed Dry Run

Input:

numbers = [2,7,11,15]

target = 9

Left	Right	Value	Sum	Action
0	3	2 + 15	17	right--
0	2	2 + 11	13	right--
0	1	2 + 7	9	Found

Return:

[1,2]

Another Example

Input:

numbers = [2,3,4]

target = 6

Left	Right	Sum	Action
0	2	6	Found

Return:



[1,3]

13. Common Use Cases (Real-Life / Interview)

Pair Searching

Used in:

Transaction matching

Budget planning

Inventory combinations

Stock pair analysis

Data reconciliation

14. Common Traps (Important!)

Trap 1

Forgetting array is sorted.

Many students immediately use HashMap.

Trap 2

Using $\leq$ instead of $<$ incorrectly.

Loop condition:

$left < right$

Trap 3

Returning values instead of indices.

Need:

indices

not

numbers

15. Builds To (Related LeetCode Problems)

LC 167 Two Sum II



LC 977 Squares of Sorted Array





LC 11 Container With Most Water



LC 15 Three Sum



LC 18 Four Sum

These form the core Two Pointer roadmap.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	Too slow
HashMap	$O(n)$	$O(n)$	Good
Two Pointers	$O(n)$	$O(1)$	Optimal

Approach 1: Brute Force

for i in range(n):

 for j in range(i+1,n):

Simple but inefficient.

Approach 2: HashMap

Store visited values.

target - current

lookup.

Approach 3: Two Pointers

Uses sorted property.

Best solution.

17. Why This Solution Works (Short Intuition)

Because the array is sorted:

Moving left pointer →

increases sum

Moving right pointer ←

decreases sum

At every step we eliminate impossible pairs.



Eventually we reach the unique valid pair.

18. Variations / Follow-Up Questions

Follow-Up 1

What if array is not sorted?

Answer:

Use HashMap

(LC 1 Two Sum)

Follow-Up 2

Find all pairs.

Requires duplicate handling.

Follow-Up 3

Find triplets.

Leads to:

LC 15 Three Sum

Follow-Up 4

Find quadruplets.

Leads to:

LC 18 Four Sum

Pattern Learned

Two Pointers

Sorted Array

↓

Compare Ends

↓

Move One Pointer

↓

Eliminate Search Space

↓

$O(n)$